



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ACTIVIDAD

Práctica 1.- Modelado Entidad Relación

BASE DE DATOS

Profesor: Gabriel Hurtado Avilés

Alumnos:

Sánchez Rodriguez Santiago

Casasola Avalos Diana Valeria

Grupo: 3CV2

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación.

1.-¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?

Un sistema de control de versiones o VCS es una herramienta que sirve para hacer un seguimiento y gestionar cambios relacionados en códigos u otros archivos a lo largo del tiempo, lo que permite a los usuarios tener un historial completo, comparar diferencias y recuperar versiones anteriores de estos mismos.

Existen 4 tipos distintos de sistemas de control de versiones estos siendo:

-Distribuido: Permite a los usuarios acceder a un repositorio desde varias ubicaciones, por lo normal lo usan para desarrolladores que necesitan trabajar en proyectos desde ubicaciones distintas o desde otros dispositivos de manera remota.

-Centralizado: Aquí todos los usuarios trabajan con el mismo repositorio central, el repositorio puede ubicarse en un servidor o en el dispositivo de algún desarrollador, se suele usar en proyectos que se necesita compartir código y ver un seguimiento en los cambios.

-Basado en bloqueos: Usa bloqueo de archivos para gestionar el acceso simultáneo a archivos y recursos, al bloquear los archivos evita que dos o más usuarios realicen cambios en conflicto en el mismo archivo o recurso.

-Optimista: En este tipo cada usuario tiene su propio espacio de trabajo privado, si quieren compartir sus cambios deben enviar una solicitud al servidor, este mismo analiza la solicitud y los cambios para ver cuales funcionan de una manera segura.

Y de problemas concreto la verdad es que existen unos varios muy importantes, como que mantiene un archivo único donde cada cambio queda guardado en fecha y autor para así evitar que se dupliquen o confundan con los nombres. O también evita que se superpongan trabajos de manera que si dos trabajadores editan un código al mismo tiempo en la misma parte del código ya no solo tome en cuenta al último en guardar si no que va a trabajar de manera inteligente para poder meter el trabajo de ambos y hacerlo funcionar.

2.- ¿Cuál es la diferencia entre Git y GitHub? No son sinónimos.

Git: Es una herramienta que se considera como el sistema de control de versiones, es un software libre y de código abierto, que se instala en tu computadora para registrar cada cambio que haces en tus archivos sin estar conectado a internet., es como si fuera un historial de tus documentos.

GitHub: Es una plataforma que ofrece un servicio de alojamiento en la nube para repositorios de Git, funciona como punto de encuentro en la red y colaboraciones para equipos que quieran subir sus códigos, también les deja revisar que cambios hacen, gestionar tareas y trabajar desde la red. Es como un Google drive que te facilita el manejar trabajos en equipo desde cualquier lugar.

3.-Defina, con sus palabras y con un ejemplo:

Repositorio: Es un espacio de almacenamiento virtual, como cuando Git guarda todos los archivos de un proyecto con un historial de cambios. Por ejemplo una carpeta que guarda un proyecto mío con código de una página que cree me va a mostrar cada modificación que se haga en esa carpeta y el código.

Confirmación (commit): Es la prueba oficial de que hicieron modificaciones en el código y su explicación del porque se hicieron los cambios, también qué es como congelar tu avance que ya funciona. Por ejemplo si le agrego algo a mi pagina web que cree y son varias cosas las que quiero ir agregando primero voy a verificar que lo que cambie si funciona y eso le hare un commit para que lo siguiente que haga o quiera investigar ya no afecte a lo que ya hice que si funciona.

Rama (branch): Es como un espacio donde puedes hacer prueba y error sin necesidad de exponer tu código original a que llegue a fallar por cambiar cosas porque aísla lo que vas a probar. Por ejemplo le agrego una función de inicio de sesión a mi pagina y para probar que funciona y no afecta el resto de mi código lo voy a aislar para probar que funcione antes de agregarlo al código.

Fusión (merge): Surve para unir las ramas independientes o aisladas a el código principal o también a otro proyecto independiente. Siguiendo con el anterior ejemplo con l inicio de sesión de pagina agregarlo a el código principal una vez que ya fue probado y aprobado.

Conflicto de fusión: Es como un bloqueo que ocurre cuando dos o mas personas modifican la misma parte de un código al mismo tiempo y Git no puede decidir con cual se va a quedar. Por ejemplo yo cambio un precio y le pongo que vale 10 pero mi compañero lo cambio al mismo tiempo a 15 entonces se produce un bloqueo en el cual uno de nosotros dos debe de resolver el bloqueo de manera manual.

Pull Request: Es como una solicitud a otros desarrolladores para que revisen y acepten o nieguen tus cambios a el código antes de agregarlos al código oficial. Por ejemplo envié un Pull Request a mi equipo para que revisen si el cambio que hice es correcto y sirve para meterlo a el código.

Archivo .gitignore: Es un texto que indica a Git que archivos o carpetas debe ignorar y tampoco rastrearlo en el historial. Por ejemplo en ese texto añadir los archivos temporales, algunas credenciales privadas o carpetas de dependencias pesadas para que no se suban a al repositorio público.

Archivo README: Es como un documento de presentación que lo más común es que se hace en formato Mark Down para explicar que hace el código, como se instala o ocupa y como pueden llegar a colaborar con él. Por ejemplo un archivo README.md que esta en la gran mayoría de repositorios de GitHub te indican que requisitos debería tener tu computadora o comandos de inicio rápido.

4.- En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

El flujo de trabajo basado en ramas es una estrategia donde la versión que es oficial se mantiene protegida, mientras que todo el otro trabajo se realizara en líneas paralelas e independientes que se les conocerán como ramas. Gracias a esta estrategia ningún desarrollador edita directamente el código principal en cada modificación que se haga se seguirán unos pasos siempre que se serán: Bifurcación, crea una nueva rama a partir de la versión estable actual. Desarrollo aislado, creas tu código en tu rama personal para no correr el riesgo de dañar el código que todos están ocupando. Y Propuesta de Integración, al final se pide una revisión para que los demás evalúen el trabajo antes de meterlo a el código principal.

La revisión por partes trabaja como filtro de calidad para el desarrollo de proyectos por varios motivos entre ellos: Detección temprana de errores, Transferencia de el conocimiento, Mantenimiento de calidad y Prevención de riesgos.

Parte B. Práctica.

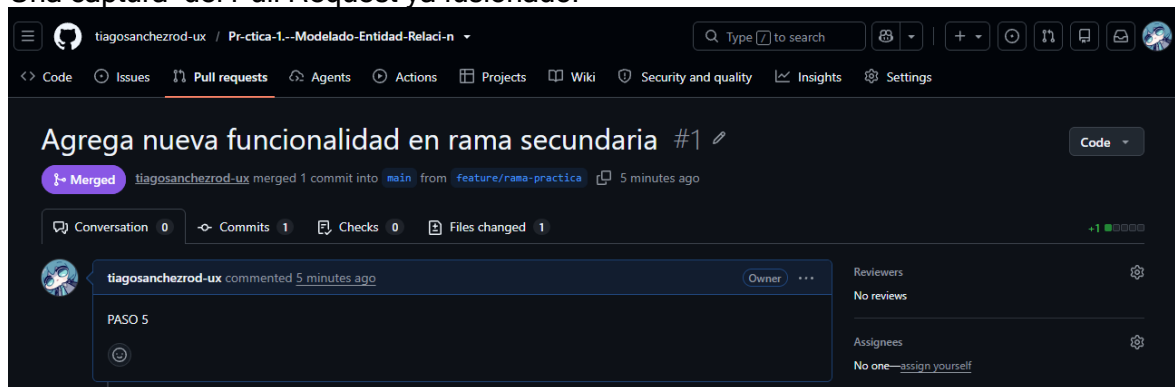
La dirección del repositorio:

<https://github.com/tiagosanchezrod-ux/Pr-ctica-1.--Modelado-Entidad-Relaci-n/pull/1>

Una captura de la salida de *git log --oneline --graph --all*:

```
Santiago@DESKTOP-Q6AE4TO MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ git log --oneline --graph --all
* ba2fd6e (origin/feature/rama-practica, feature/rama-practica) Agrega nueva funcionalidad en rama s
ecundaria
* bcc7532 (HEAD -> main, origin/main) Actualiza el README con notas finales de la práctica
* db44fd8 Actualiza el archivo de código base con lógica adicional
* 79c712a Agrega archivo de código base inicial para el desarrollo
* e6491f2 Configura el archivo .gitignore para excluir binarios y temporales
* 4d6cddf Añade archivo README con datos personales, índice y contenido de la práctica
```

Una captura del Pull Request ya fusionado:



Ejercicio 2. El sistema gestor en un contenedor: Docker

Parte A. Investigación.

1.- Qué es un contenedor y en qué se diferencia de una máquina virtual, en términos de arranque, tamaño y aislamiento.

Un contenedor es una unidad de software que empaqueta el código fuente, las bibliotecas y todo lo necesario para que una aplicación se ejecute de manera correcta e igual en cualquier infraestructura.

DIFERENCIAS ENTRE CONTENEDORES Y MÁQUINAS VIRTUALES:

-ARRANQUE: En un contenedor lo más común es que se tarde milisegundos o segundos porque no tiene que arrancar todo el sistema operativo completo. En cambio, una máquina virtual que se tarda minutos porque debe iniciar todo el hardware, aparte del sistema operativo.

-TAMAÑO: Un contenedor solo llega a ocupar MB porque las imágenes que llega a tener solo son la aplicación y sus dependencias. Por otro lado, las máquinas virtuales lo más común es que usen GB porque lleva todo un sistema operativo encima, con sus bibliotecas y archivos del sistema.

-AISLAMIENTO: Un contenedor usara mecanismos del kernel para aislar procesos y cgroups para limitar recursos tanto de memoria como de CPU. Y la máquina virtual utilizara un hipervisor para emular hardware físico para así garantizar seguridad entre VMs.

2.- Defina:

-IMAGEN: Es una plantilla y de solo lectura que guarda el código de la aplicación, sus bibliotecas, las dependencias y los binarios que necesita para ejecutar el software, es como si fuera un plano de construcción donde tiene las instancias en ejecución.

-CONTENEDOR: Trabaja como un entorno aislado y ligero que ejecuta la aplicación con sus procesos y lo hace compartiendo el núcleo del sistema mediante namespaces y cgroups.

-VOLUMEN: Es un mecanismo de persistencia de datos administrado por contenedores que permiten almacenar información fuera del estado del contenedor, lo que se guarde en un volumen se conservarán, aunque se elimine o se vuelva a crear un contenedor, esto facilitando el almacenamiento de bases de datos y archivos de configuración.

-PUERTO PUBLICADO: Es un mapeo de red que muestra un puerto interno del contenedor hacia un puerto específico en la máquina host, para que el tráfico de red externo o de otros servicios puedan acceder a la aplicación que está dentro del contenedor aislado.

3.- Por qué el volumen es indispensable y qué ocurre exactamente si no lo declara.

Es indispensable porque no tiene una dependencia entre el ciclo de vida de los datos y del contenedor. Porque de normal cuando creas un contenedor este crea una capa de escritura temporal donde se van a almacenar cualquier tipo de archivos nuevos o modificados, el problema con esa capa es que cuando el contenedor se elimine o se reconstruya toda esa información que guardo se va a borrar de manera permanente, eso no pasa con el volumen porque almacena la información conectándola directamente a los archivos host o a un almacenamiento independiente sin importar el estado del contenedor.

Si no se llega a declarar un volumen las consecuencias son directas, por ejemplo cualquier base de datos que corra dentro del contenedor va a desaparecer y con eso el servicio no sería persistente, también todas las operaciones de escritura se acumularan en la capa y eso consume una gran cantidad de espacio en el disco, además si no se define imposibilita compartir información con otros contenedores, también complica la creación de una copia de seguridad porque haría que recurrir a extracciones manuales desde el contenedor antes de ser destruido. Todo eso te ahorra declarar un volumen que guardara la información para que duren los datos sin importar el estado del contenedor.

Parte B. Práctica.

El archivo `compose.yaml` versionado:

tiagosanchezrod-ux / Pr-ctica-1.--Modelado-Entidad-Relaci-n

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Pr-ctica-1.--Modelado-Entidad-Relaci-n Public

main 2 Branches 0 Tags

Go to file

Add file Code

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published

tiagosanchezrod-ux Agrega configuracion de PostgreSQL en compose.yaml c3ff779 · 35 minutes ago 8 Commits

.gitignore	Configura el archivo .gitignore para excluir binarios y tempor...	yesterday
README.md	Actualiza el README con notas finales de la práctica	yesterday
codigo_base.txt	Agrega nueva funcionalidad en rama secundaria	yesterday
compose.yaml	Agrega configuracion de PostgreSQL en compose.yaml	35 minutes ago

README

Práctica 1: Control de Versiones

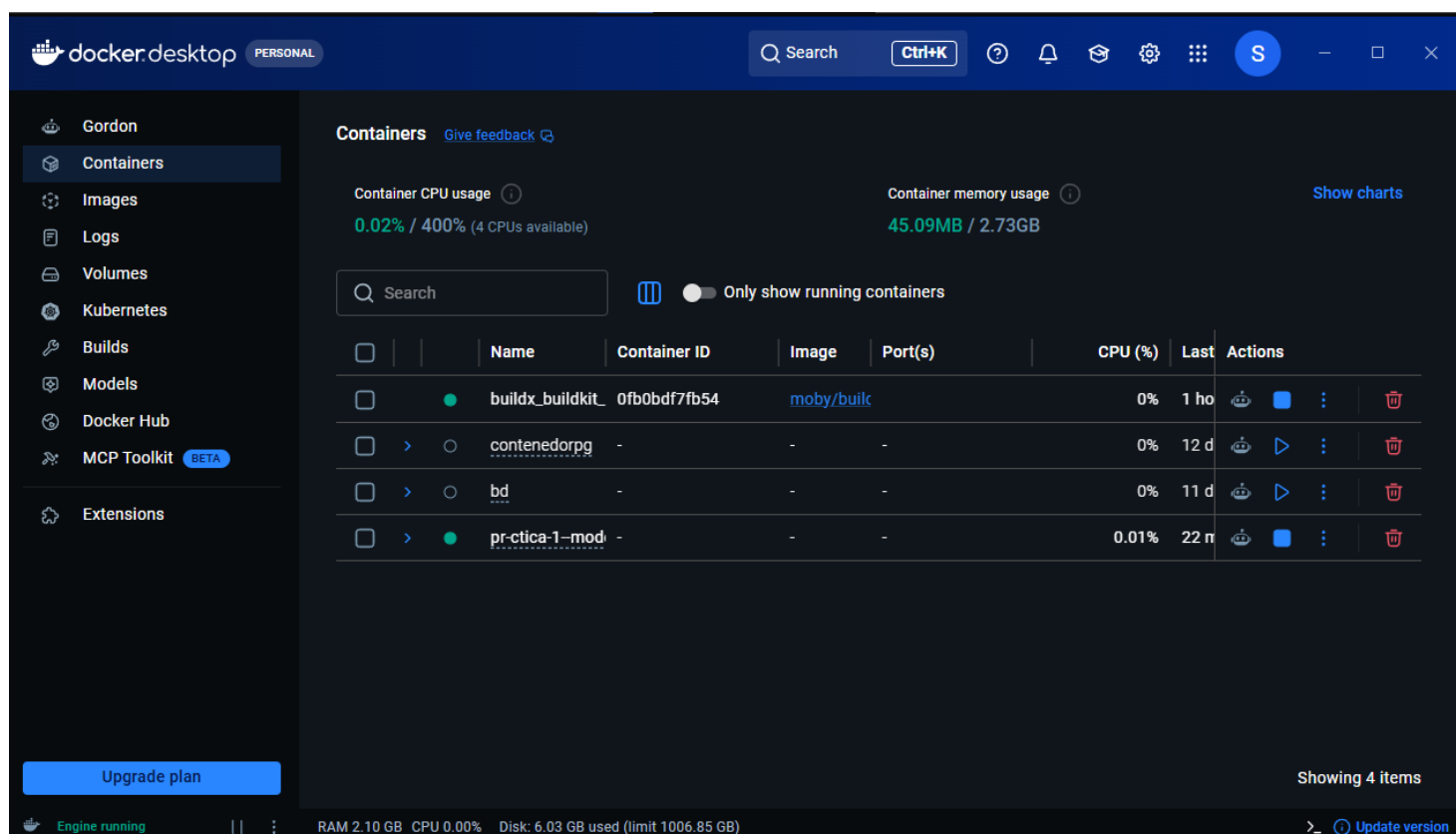
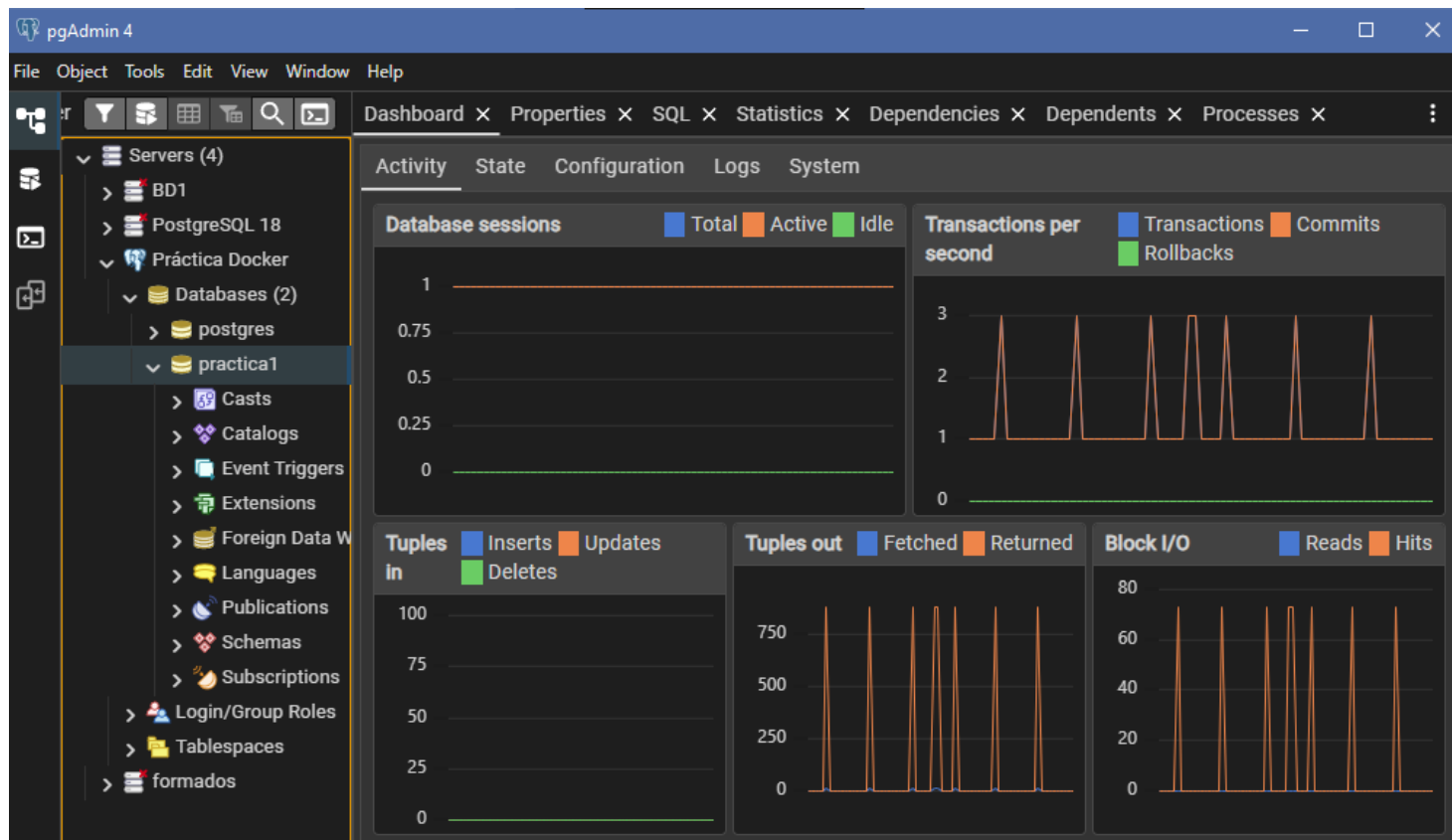
compose.yaml Agrega configuracion de PostgreSQL en compose.yaml 35 minutes ago

```
Santiago@DESKTOP-Q6AE4T0 MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ git add compose.yaml
```

```
Santiago@DESKTOP-Q6AE4T0 MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ git commit -m "Agrega configuracion de PostgreSQL en compose.yaml"
[main 9b40d7d] Agrega configuracion de PostgreSQL en compose.yaml
1 file changed, 16 insertions(+)
create mode 100644 compose.yaml
```

```
Santiago@DESKTOP-Q6AE4T0 MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 489 bytes | 489.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/tiagosanchezrod-ux/Pr-ctica-1.--Modelado-Entidad-Relaci-n.git
fd60f35..c3ff779 main -> main
```

Una captura de la conexión al gestor:



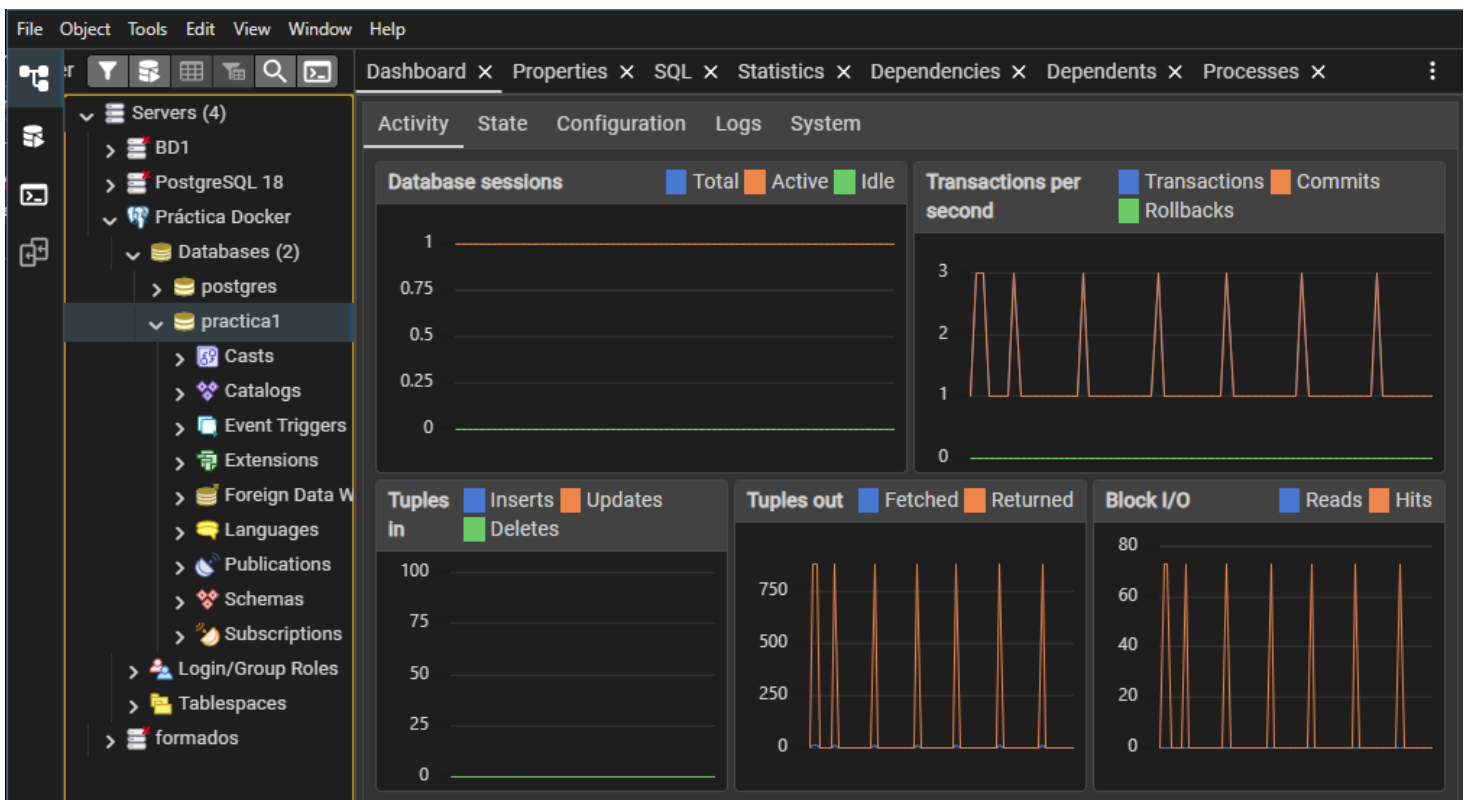
Una captura de la prueba de persistencia:

```

Santiago@DESKTOP-Q6AE4TO MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ docker compose down
time="2026-09-13T23:06:43-06:00" level=warning msg="C:\\Users\\Santiago\\Pr-ctica-1.--Modelado-Entid
ad-Relaci-n\\compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it
to avoid potential confusion"
[+] down 2/2
✓ Container pg-practical Removed 2.0s
✓ Network pr-ctica-1--modelado-entidad-relaci-n_default Removed 0.4s

Santiago@DESKTOP-Q6AE4TO MINGW64 ~/Pr-ctica-1.--Modelado-Entidad-Relaci-n (main)
$ docker compose up -d
time="2026-09-13T23:07:09-06:00" level=warning msg="C:\\Users\\Santiago\\Pr-ctica-1.--Modelado-Entid
ad-Relaci-n\\compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it
to avoid potential confusion"
[+] up 2/2
✓ Network pr-ctica-1--modelado-entidad-relaci-n_default Created 0.1s
✓ Container pg-practical Started 1.4s

```



Ejercicio 4. Estado del arte: tres artículos científicos

Artículo 1: SRank: Guiding schema selection in NoSQL document stores.

1.- Cita en formato APA:

Sachdeva, S., Bansal, N., & Bansal, H. (2024). SRank: Guiding schema selection in NoSQL document stores. Data & Knowledge Engineering, 154, 102360. <https://doi.org/10.1016/j.datak.2024.102360>.

2.- Problema que aborda:

El problema que se muestra es que aunque la flexibilidad de esquemas en las bases de datos NoSQL acelera el desarrollo inicial, si se elige un diseño incorrecto afecta negativamente el rendimiento, los costos de acceso y el mantenimiento de la aplicación.

3.- El método o la propuesta de los autores:

Ellos proponen un modelo de cuatro fases que toma como entrada un modelo Entidad-Relación y consultas de

carga de trabajo, con esos datos el sistema crea alternativas de esquemas y grafos de consultas, calcula una métrica de clasificación llamada SRank evaluando grafos y rutas para ver cual es el mejor diseño posible y más optima.

4.- El resultado principal que reportan:

A partir de un caso de estudio de un sistema de reservaciones de hotel y una evaluación experimental, validaron que los valores de SRank se alinean perfectamente con las métricas reales de rendimiento como el tiempo de respuesta y almacenamiento, con esto haciendo más rápido la selección del esquema óptimo.

5.- Con qué tema de la Unidad Temática I se relaciona:

Se relaciona con el Tema 1.1.1 (Características) en como evalúan el rendimiento y almacenamiento en las bases de datos, también con el tema 1.4 (Modelos de datos) con su subtema 1.4.2 (Clasificación) porque analizan y estructuran bases de datos NoSQL no relacionadas.

6.- Qué le aporta a usted para su proyecto del curso:

Me aporta una manera más avanzada de ver cómo la selección estructurada de esquemas te ahorra muchos problemas de rendimiento en arquitecturas de bases de datos de la actualidad y aplicando lo de este artículo incluso ahorre tiempo para la selección de esquemas óptimos.

Artículo 2: A systematic review of database design in the development and usage of floods early warning systems.

1.- Cita en formato APA:

Chinguwo, D. D., & Deus, D. (2025). A systematic review of database design in the development and usage of floods early warning systems. *African Journal on Land Policy & Geospatial Sciences*, 8(8), 1312-1324.
<https://doi.org/10.48346/IMIST.PRSM/ajlp-gs.v8i8.59222>

2.- Problema que aborda:

Trata sobre la falta de atención que se le da al diseño estructurado de las bases de datos durante el desarrollo de sistemas de alerta temprana de inundaciones, lo que puede afectar en gran medida la efectividad, el almacenamiento de información y la forma de trabajar de dichos sistemas.

3.- El método o la propuesta de los autores:

Ellos propusieron e hicieron una revisión sistemática de la literatura guiada por el marco metodológico PRISMA. Definieron criterios estrictos tanto de inclusión como de exclusión y buscaron en bases de datos científicas como Research4life y Wiley Online, con publicaciones en inglés de los años 2002-2022 mediante palabras clave relacionadas con modelos, diseño y desarrollo de bases de datos para alertas tempranas.

4.- El resultado principal que reportan:

De 3031 manuscritos identificados al inicio, 19 de esos artículos enfocados en el diseño e implementación de esos sistemas, eligieron solo 12 estudios clave para un análisis cualitativo detallado sobre el impacto del diseño de bases de datos en el rendimiento de los sistemas de alerta.

5.- Con qué tema de la Unidad Temática I se relaciona:

Se relaciona con los subtemas 1.1.4 (Ciclos de vida) al demostrar la importancia de la planificación y diseño de una base de datos dentro de un ciclo de desarrollo en un sistema de información y también con el subtema 1.4.1 (Modelos de datos) se ve como un modelado y diseño adecuado de los datos sirve como molde para un funcionamiento adecuado en su estructura.

6.- Qué le aporta a usted para su proyecto del curso:

Me deja claro el por qué la fase de diseño de la base de datos es tan importante y no debe de saltarse ni tomarla a la ligera, me ayuda a mostrándome de manera detallada la manera de planificación del almacenamiento de información y el modelo conceptual desde un inicio del proyecto.

Artículo 3: Difficulties in understanding the transition between database design stages: An experiment from a semantic distance perspective.

1.- Cita en formato APA:

Shin, S.-S., Lin, Y.-S., & Chen, Y.-C. (2025). Difficulties in understanding the transition between database design stages: An experiment from a semantic distance perspective. *Journal of Engineering Education*, 114(4), 1-19. <https://doi.org/10.1002/jee.70038>

2.- Problema que aborda:

Se muestra la dificultad que enfrentan los estudiantes de bases de datos para entender y traducir las necesidades de un negocio a modelos Entidad-Relación y posteriormente a modelos relacionales, debido a la brecha o también llamada distancia semántica que están entre las distintas etapas del diseño.

3.- El método o la propuesta de los autores:

Su método fue que, utilizando la teoría de redes semánticas como su marco analítico, lograron diseñar un experimento con estudiantes de bases de datos. Los estudiantes realizaron dos tareas consecutivas de traducción: pasar de los requerimientos de negocio a modelos Entidad-Relación y de modelos Entidad-Relación a modelos relacionales, después de calificar su desempeño mediante qué tan efectivo fue y que tanta eficiencia tenían en la resolución de problemas.

4.- El resultado principal que reportan:

Los resultados mostraron una efectividad y eficiencia muchísimo mayores al convertir modelos Entidad-Relación a modelos relacionales, que cuando se debe traducir requerimientos de negocio a modelos Entidad-Relación. Esto demostró que el salto conceptual desde los requerimientos hacia el modelo Entidad-Relación representa una distancia semántica mayor y requiere un esfuerzo de elaboración mucho más complejo.

5.- Con qué tema de la Unidad Temática I se relaciona:

Se relaciona con el tema 1.4 (Modelo de datos) en su subtema 1.4.1(Importancia) y 1.4.2(Clasificación) al ver cómo se construyen y como deben entenderse los modelos de Entidad-Relación y los relacionales para saber cómo funciona la estructura de la relación. y el subtema 1.1.4(Ciclo de vida) que muestra las fases del ciclo de desarrollo de una base de datos desde los requisitos hasta su diseño lógico y conceptual.

6.- Qué le aporta a usted para su proyecto del curso:

Me ayuda a entender mejor de los errores comunes al entender y traducir reglas de negocio hacia esquemas formales. Me muestra porque es bueno el prestar atención a la identificación de entidades, atributos y relaciones desde la fase de análisis de requerimientos, para asegurar un diseño bien fundamentado desde el inicio del proyecto y no tener problemas en un futuro.

BIBLIOGRAFIA

Ejercicio 1:

- GitLab. (s. f.). ¿Qué es el control de versiones? <https://about.gitlab.com/es/topics/version-control/>
- Aprender21. (s. f.). ¿Qué es Git y GitHub? Fundamentos del control de versiones para desarrolladores. <https://www.aprender21.mx/blog/que-es-git-github-version-control>
- Atlassian. (s. f.). Flujo de trabajo de ramas de función en Git. <https://www.atlassian.com/es/git/tutorials/comparing-workflows/feature-branch-workflow>

Ejercicio 2:

- Amazon Web Services. (s. f.). ¿Cuál es la diferencia entre los contenedores y las máquinas virtuales?

<https://aws.amazon.com/es/compare/the-difference-between-containers-and-virtual-machines/>

- Sharma, M. (2025, 17 de noviembre). ¿Qué es un volumen Docker? DEV Community.

<https://dev.to/meghasharmaaaa/what-is-docker-volume-4km>